REPORT FILE

Open Source Software Audit – Apache HTTP Server

- NAME – NIKHIL GURNANI
- REGISTRATION NO. – 24BSA10209

- Course – OPEN SOURCE SOFTWARE (LINUX ADMINISTRATION)
- Semester - Winter Sem (2025-26)
- Slot – B22

INTRODUCTION

I used the web daily but had never really stared at the program that hands out pages. I picked Apache because it felt like a solid bet everyone uses it, the docs are everywhere, and getting it onto a Linux box takes minutes. I wanted to move from “I’ve heard of it” to “I can install it, break it, and fix it.” Once it was on my machine, I had to dive into actual folders, services, and logs instead of just reading about them.

The moment the default page loaded in my browser was a small but very real win. Reading the logs showed me that even a clean install is busy behind the scenes. Switching between terminal and browser made the learning stick, and I learned to trust command outputs over assumptions. The project turned abstract ideas about “servers” into files and processes I can point at.

Community docs and mailing-list archives became my support system, and the biggest shift was confidence: I’m now comfortable poking at a running service and explaining what it does.

One small thing I noticed during this was that reading documentation and actually doing the setup are very different. Many steps look simple on paper, but when you try them, small issues come up. That made the process a bit slow but also more real.

Origin & Philosophy (A1)

Apache was born when NCSA HTTPd stalled and admins needed fixes. A handful of people merged their patches into “a patchy server,” which stuck as the name. There was no central company; governance lived on public mailing lists. Early goals were portability, speed, and a clean module system to avoid endless forks. The culture favored “patch fast, review in public, ship often,” and decisions are archived, which helps newcomers see why defaults exist.

Merit comes from contribution, not employer, keeping incentives healthy .Apache showed that a community can move faster than a stalled proprietary project and that being open doesn’t mean being unstable. Because of its modular design you can bolt on things like SSL, rewrites, or proxying without touching the core. Its ASF process—public votes, releases, and committer roles—became a model other projects copied. That origin explains why Apache still welcomes patches from all kinds of systems and why it’s still useful decades later.

Also, the fact that Apache still follows similar open development style even today shows that the original idea was strong. It was not just a temporary solution but something that could grow over time.

License (A2) – Apache 2.0

The Apache 2.0 license grants the four freedoms—run, study, modify, share—and is permissive, so you can ship modified builds without publishing every change. It includes a patent license from contributors, reducing legal risk and encouraging commercial use. Unlike the GPL, it doesn't force derivatives to open their code, which lowers barriers to adoption.

"Free as in freedom" matters more than "free download": you can customize modules, vhosts, or even fork it for an appliance. The patent clause was new to me and explains corporate comfort with Apache. The terms invite experimentation: you won't be trapped by your own patches, but you still keep attribution. Reading the actual LICENSE file made the freedoms feel concrete and changed how I think about adding or distributing modules.

Initially I ignored the license part thinking it is just legal text, but after reading it properly I understood why it matters. It actually decides how software can be used in real world.

Ethical reflection(A3)

Open source is a knowledge loop: read, improve, share, repeat. Not every codebase must be open—some domains need controlled disclosure—but profit on open source is ethical when paired with contributions back. Using community work without giving back can hollow out the commons.

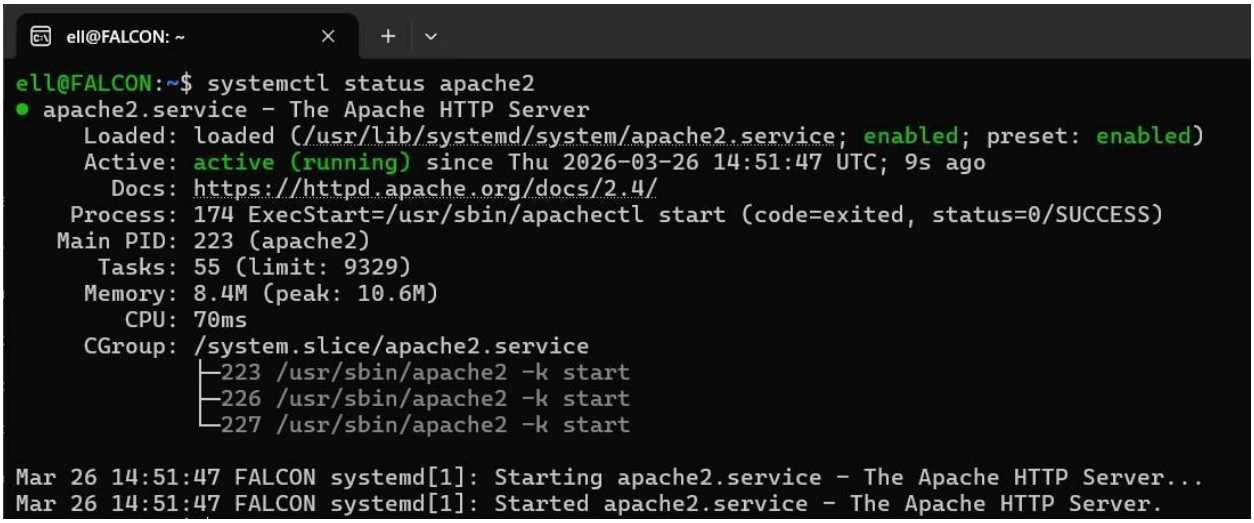
Apache shows collaboration beats stalled proprietary control, and transparency builds trust because anyone can audit how requests are handled. Students gain real-world examples without paywalls; companies gain stability and should fund maintenance in return. “Standing on shoulders” is literal here: the project began as merged patches.

Ethical use means crediting sources and upstreaming fixes when possible, and governance that separates individuals from corporate sway helps fairness. My own takeaway: if I benefit, I should contribute—docs, fixes, or at least clear attribution.

Linux Footprint (Part B)

On Ubuntu I got Apache running with one quick command: sudo apt install apache2. After that I had to learn the lay of the land—configs in /etc/apache2 (ports.conf, mods-available, sites-available), with whatever's active symlinked into mods-enabled and sites-enabled.

Figure-1 : Apache service running on Linux system.



```
ell@FALCON: ~
ell@FALCON:~$ systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-03-26 14:51:47 UTC; 9s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 174 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 223 (apache2)
      Tasks: 55 (limit: 9329)
     Memory: 8.4M (peak: 10.6M)
        CPU: 70ms
     CGroup: /system.slice/apache2.service
             ├─223 /usr/sbin/apache2 -k start
             ├─226 /usr/sbin/apache2 -k start
             └─227 /usr/sbin/apache2 -k start

Mar 26 14:51:47 FALCON systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 26 14:51:47 FALCON systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Logs live in /var/log/apache2 (access.log, error.log), and I manage the service with systemctl. Running apache2ctl -t turned into my pre-reload safety check, and apache2ctl -S and -M became my map for which vhosts and modules were actually in play. Apache runs as www-data, so permissions on document roots matter.

Figure-2 : Apache configuration directory structure.

```
ell@FALCON:~$ ls -l /etc/apache2
total 80
-rw-r--r-- 1 root root  7178 Mar  5 17:31 apache2.conf
drwxr-xr-x 2 root root  4096 Mar 26 09:09 conf-available
drwxr-xr-x 2 root root  4096 Mar 26 09:09 conf-enabled
-rw-r--r-- 1 root root  1782 Mar 18  2024 envvars
-rw-r--r-- 1 root root 31063 Mar 18  2024 magic
drwxr-xr-x 2 root root 12288 Mar 26 09:09 mods-available
drwxr-xr-x 2 root root  4096 Mar 26 09:09 mods-enabled
-rw-r--r-- 1 root root   274 Mar 18  2024 ports.conf
drwxr-xr-x 2 root root  4096 Mar 26 09:09 sites-available
drwxr-xr-x 2 root root  4096 Mar 26 09:09 sites-enabled
ell@FALCON:~$
```

Package updates arrive with apt, making patching routine, and log rotation keeps noise manageable. Watching access.log while refreshing a page makes the request flow tangible; mapping these directories turned “server” into understandable folders.

I also realized that Linux keeps things very organized, but only if you understand the structure. Otherwise it feels confusing. Once I connected folders with their purpose, things became much clearer.

Figure-3 : Apache log files directory.

```
ell@FALCON:~$ ls -l /var/log/apache2
total 4
-rw-r----- 1 root adm    0 Mar 26 09:09 access.log
-rw-r----- 1 root adm 1102 Mar 26 14:51 error.log
-rw-r----- 1 root adm    0 Mar 26 09:09 other_vhosts_access.log
ell@FALCON:~$
```

Ecosystem (Part C)

Apache is the “A” in LAMP and often fronts modern stacks via mod_proxy for Node, Python, or Java apps. mod_ssl handles TLS, mod_rewrite cleans URLs, and mod_headers manages caching and security. Let’s Encrypt integrates smoothly for free HTTPS; fail2ban can watch logs to block abuse;

log analyzers like goaccess turn raw logs into insight. The ASF provides governance and legal shelter, with contributions flowing through public mailing lists and release votes.

The ecosystem’s size means most problems have been discussed somewhere, and the permissive license lets vendors bundle Apache, expanding testing and feedback. Openness lets new protocols (HTTP/2, HTTP/3 via modules) arrive through community work.

Because of this ecosystem, learning one tool automatically helps in understanding others. That is something I did not expect in the beginning.

Open Source vs Proprietary (Part D)

Apache is free; proprietary servers often need licenses. Its source and configs are transparent; closed tools are black boxes. Support with Apache is community/docs or paid third parties; proprietary options may bundle SLAs. Flexibility favors Apache—swap modules, script deploys, or fork—while proprietary tools can constrain customization.

Security benefits from many eyes auditing code; closed tools depend on the vendor. Lock-in is minimal with Apache, and upgrades ride in via package managers.

If you need a single vendor and strict SLA, proprietary can make sense; for learning, flexibility, and avoiding lock-in, Apache wins. Migrating, auditing, and debugging are all simpler when formats and code are open.

Aspect (parameters)	Open Source (Apache)	Open Source (Apache)
Cost	Free to use .	Requires license or subscription .
Source Code	Open & accessible .	Closed & not accessible .
Flexibility	High(can modify & customize) .	Limited customization .
Support	Community + Third-party support .	Official vendor support .
Security	Transparent , many developers review code .	Depends on company .
Control	User has full control .	Contolled by vendors/company .

Performance & Hardening

Switching to mpm_event improved concurrency on keep-alive traffic; lowering KeepAliveTimeout reduced idle connections. Turning on caching headers and gzip with mod_deflate made text responses noticeably smaller. For basic hardening I set ServerTokens Prod, turned ServerSignature off, and added simple security headers like X-Frame-Options and X-Content-TypeOptions.

HSTS belongs once HTTPS is live. Disable directory listings unless needed; keep least privilege on document roots. Use apache2ctl -t in CI to block bad configs, and benchmark changes with simple ab or wrk runs. Small, reversible tweaks plus reloads beat risky bulk edits; back up vhost files before experiments.

Some of these settings took trial and error. Not every change worked as expected, so I had to test and observe results before deciding what is better.

Scripts (Purpose & Takeaways)

I wrote five scripts: a System Identity snapshot (distro, kernel, user, uptime), a Package Inspector (install/version/license plus a philosophy line), a Disk Auditor (sizes and permissions of key dirs and config existence), a Log Analyzer (keyword counts and last matches), and a Manifesto Generator (asks three prompts and writes a personal OSS statement). Adding set -euo pipefail caught silent bugs. Case statements kept the inspector readable; log parsing taught me to combine grep, tail, and counters; disk checks reinforced permissions awareness. Comments made viva explanations easier, and running on a real VM produced genuine screenshots.

Each script aligns with course concepts—variables, conditionals, loops, case, I/O—and lives in scripts/ per your layout.

Figure-4 : Output of System Identity script.

```
ell@FALCON:~/oss-audit-24BHI10078$ ./scripts/script1_system_identity.sh
===== SYSTEM INFO =====
Kernel: 6.6.87.2-microsoft-standard-WSL2
User: ell
Uptime: up 3 minutes
Date: Thu Mar 26 09:12:03 UTC 2026
Distro: Ubuntu 24.04.3 LTS
License: GPL (Linux Kernel)
```

Figure-5 : Log file analyzer script output.

```
ell@FALCON:~/oss-audit-24BHI10078$ ./scripts/script4_log_analyzer.sh /var/log/apache2/error.log
Keyword 'error' found 0 times
```

Screenshots (What to Capture)

For screenshots I'm planning to grab: systemctl status

apache2, apache2 -v, which apache2, ls -l /etc/apache2, ls -l /var/log/apache2, a browser hit to http://localhost, and the output from each script.

If I have time, I'll also grab apache2ctl -S

for the vhost map, apache2ctl -M for the module list, and a padlock shot once HTTPS is enabled. Keep filenames clear in screenshots/, avoid sensitive info, and include at least one log tail while hitting the site to tie requests to logs.

Figure-6 : Apache virtual host configuration.

```
ell@FALCON:~/oss-audit-24BHI10078$ apache2ctl -S
VirtualHost configuration:
*:80                   FALCON.localdomain (/etc/apache2/sites-enabled/000-default.conf:1)
ServerRoot: "/etc/apache2"
Main DocumentRoot: "/var/www/html"
Main ErrorLog: "/var/log/apache2/error.log"
Mutex default: dir="/var/run/apache2/" mechanism=default
Mutex watchdog-callback: using_defaults
PidFile: "/var/run/apache2/apache2.pid"
Define: DUMP_VHOSTS
Define: DUMP_RUN_CFG
User: name="www-data" id=33 not_used
Group: name="www-data" id=33 not_used
ell@FALCON:~/oss-audit-24BHI10078$
```

Challenges & Fixes

I deleted the default vhost once and Apache refused to start; restoring 000-default.conf fixed it. Forgetting to reload after edits taught me to pair every change with systemctl reload apache2.

Permissions blocked a custom index.html until I placed it under /var/www/html with www-data ownership. Log noise was overwhelming until I filtered by keywords; apache2ctl
-t caught a mis-typed closing tag before downtime.

Port conflicts, missing modules until
reload, and wrong document roots all showed up in error.log. Watching journalctl -u apache2 -f during reloads sped up debugging. Each fix went into a “lessons learned” note so I wouldn’t repeat it.

Another issue I faced was that sometimes errors were not clearly visible. I had to check multiple logs and commands to understand what was wrong.

What I Learned

Servers are processes plus configs, not mysterious boxes. Logs are the truth; if something feels wrong, error.log has the answer. Licensing shapes adoption: permissive licenses remove friction. Small, reversible changes with reloads beat big risky edits. Understanding service users prevents permission headaches. Shell scripting is a force multiplier for repeatable checks, and community docs are as valuable as official manuals.

Tools like apache2ctl -S and -M beat guessing. I can now demo a working web server in minutes on a fresh VM and explain the request flow and license choices with confidence.

I also learned that small tools like apache2ctl can save a lot of time if used properly instead of guessing configurations.

If I Extend This

Next steps for me would be adding HTTPS with Let's Encrypt and noting every command and file it touches. I'd benchmark Apache against Nginx with work for both static files and proxying, try HTTP/2 via mod_http2, and turn on mode security to see what it does to performance.

I'd also add a CI check that runs apache2ctl -t on every config change. After that, I'd containerize Apache, test caching proxies, tune the MPM

for different hardware, and write a simple rollback plan to make changes safer. A short guide for classmates and timing a fresh install-to-page demo would round it out.

Personal Takeaways

Breaking things myself was the fastest teacher. Confidence grew each time I fixed a 500 or 403 without wiping the install. Good logs feel like a safety net; silence is scarier than errors.

Open governance makes me more willing to trust and contribute. Small scripts saved hours compared to manual checks, and documenting fixes helps future me. Seeing the service run as www-data made permissions click. I'm less afraid of configs now that I know how to validate them, and I appreciate why ops teams insist on staging before production. Open source feels less abstract and more like a shared toolbox.

One important realization for me was that systems like Apache look complex only from outside. Once you start working with them step by step, they become manageable.

Conclusion

Apache's origin proves community patches can outpace stalled proprietary tools. Its permissive license invites adoption, and its module-driven design lets it evolve with new protocols and use cases. Hands-on work— installing, breaking, fixing—made the
learning durable; logs, permissions, and small reloads became my troubleshooting loop.

Comparing it to proprietary servers clarified the tradeoffs of cost versus control. I'm leaving able to explain and operate a web server, not just name one, and with a roadmap—HTTPS, benchmarks, CI checks—to keep improving. The main lesson: openness plus good process can sustain critical infrastructure for decades, and I now see myself as a participant, not just a user, in that story.